

generator should have T&& reference_type

Document #: P2529R0
Date: 2022-01-25
Project: Programming Language C++
Audience: Library Evolution Working Group
Reply-to: Mathias Stearn
<redbeard0531@gmail.com>

Contents

- [1 Introduction](#)
- [2 Starting point](#)
 - [2.1 Things that we \(probably\) all agree about](#)
 - [2.2 Major point of disagreement](#)
- [3 Justification](#)
 - [3.1 Performance and move-only type support](#)
 - [3.2 Consistency with the rest of the language.](#)
 - [3.3 Best match for the generator<T> use cases](#)
 - [3.4 Simplicity](#)
- [4 Responses to objections](#)
 - [4.1 Returning an rvalue reference from `\*it` violates `input\_or\_output\_iterator` semantic requirements](#)
 - [4.2 Even if valid, rvalue-returning ranges are unsafe to consume](#)
 - [4.3 Implicitly moves in body of generator](#)
 - [4.4 Implicitly copies in body of generator](#)
 - [4.5 We have `generator<T&&>` for people that want moves and it is more explicit](#)
- [5 Example implementation](#)
- [6 References](#)

§ 1 Introduction

This paper attempts to make the case that the `generator<T>`, as proposed in [P2502R0] (and earlier in [P2168R3]), should use `T&&` as its `reference_type`. While this paper is critiquing a specific design choice in that paper, I would like to be clear that it is not intended in any way as a slight against its author. I very much appreciate the work that Casey has done both specifically on generator and on ranges in general.

Note: An updated draft of D2502R1 was sent out after this paper was completed, and I have not had time to fully process the changes and modify this paper (other than this paragraph) to reflect the update. I think the most notable change is that now `generator<T&&>` in that proposal has the `co_yield` behavior I am proposing for `generator<T>`, but *not* proposing for `generator<T&&>`. I think it is inconsistent with my model to have it for both `generator<T>` and `generator<T&&>`, since you cannot currently pass a `T` lvalue to a function taking `T&&`.

§ 2 Starting point

§ 2.1 Things that we (probably) all agree about

- Everything about `generator<U>` where `U` is `T&`, `const T&`, or `T&&`
 - `reference_type` should be `U` and the default `value_type` should be `remove_cvref<U>`
 - The signature of `promise_type::yield_value` should be `yield_value(U)`, which may restrict what can be passed to `co_yield`
- We don't care about `generator<U>` where `U` is `const T`, `const T&&` or has `volatile` anywhere
 - Perhaps some of these should *work*, but it isn't worth spending any committee time to make them *work well*
- For `generator<T>` where `T` has no cvref qualifications:
 - default `value_type` should be `T`
 - `reference_type` should NOT be `T`, `const T`, or `const T&&`
 - `T` was proposed in [P2168R3], but the authors have agreed that it is not the right choice
 - This leaves `T&`, `const T&` and `T&&` as options worth considering
 - support `co_yield` `EXPR` where `EXPR` is either an lvalue or rvalue
 - note that this is different from `generator<T&&>` which only allows yielding rvalues
 - lvalues and constant rvalues passed to `co_yield` must not be able to be mutated from outside the function
 - non-const rvalues passed to `co_yield` may be modified or at least moved from (that isn't to say that we agree that they must be, only that I do not anticipate objections to rvalues being mutated)

§ 2.2 Major point of disagreement

- For `generator<T>` where `T` has no cvref qualifications:
 - Should `reference_type` be `T&&` (this paper) or `const T&` ([P2502R0])
 - while `T&` is a valid option to discuss, I don't think anyone has proposed it for `std::generator`, even though it is existing practice for some generator implementations
 - What should we do when an lvalue is passed to `co_yield`?
 - [P2502R0] proposes to hand out a `const&` directly to it
 - This paper proposes to copy it and hand out an rvalue-reference to the copy (see the [example implementation](#) for how this works).

§ 3 Justification

§ 3.1 Performance and move-only type support

These are really two different issues, but the example code is the same for both.

make_gen() is a stand-in for a function returning generator<T>	
T&&	const T&
<pre>ranges::copy(generator, output); // no copies make_gen() to<vector>; // no copies</pre>	<pre>ranges::copy(generator, output); // copies make_gen() to<vector>; // copies make_gen() views::move to<vector>; // still copies</pre>
<pre>for (auto s : make_gen()) {} // no copies for (auto&& s : make_gen()) { consume(s); // copies (as it should!) consume(std::forward<decltype(s)> (s)); // no copies }</pre>	<pre>for (auto s : make_gen()) {} // copies for (auto&& s : make_gen()) { consume(s); // copies (as it should!) consume(std::forward<decltype(s)> (s)); // copies consume(std::move(s)); // still copies; }</pre>

If this generator produced a move-only type, then every copy above would be a compilation failure, rather than a performance issue. If the generator function was in a third-party library or code that was otherwise unable to be changed, you would be stuck.

§ 3.2 Consistency with the rest of the language.

For various reasons, using an actual non-reference T as the argument to `yield_value()` and the return from `operator*()` are not good ideas. That leaves use trying to emulate with the best analog we can.

§ 3.2.1 Inside the generator function

Inside a `generator<T>` function, I think `co_yield` `EXPR` is best thought of similarly to either `f(EXPR)` where `f` takes a `T` or `return` `EXPR` inside of a function returning `T`. In both of those cases, `EXPR` will be copied if an lvalue, but not if it is an rvalue.

For function arguments, it is generally accepted that the equivalent to `T` is to have overloads for `T&&` and `const T&`. They aren't *exactly* equivalent, since the final state of an xvalue argument depends on what the callee does with it, but that is generally ignored since it is usually a bad idea to depend on a specific state of a variable after passing it to `std::move()`. Additionally, you are able to pass immobile xvalues to `T&&/const T&` functions as long as they don't try to move or copy, while you can't even pass them to `T`-taking functions.

This is what I propose doing for `generator<T>::promise_type::yield_value()`. Unfortunately, it cannot take a `T` because the lifetime is too short. So the next best thing is to have it take a `T&&` (which

also does the right thing when `T` is a reference) and offer a `const T&` overload for copyable, non-reference `T` types that yields a copy of the passed in object.

§ 3.2.2 Consuming a generator

<code>T func()</code>	<code>generator<T> gen()</code>
<code>func().mutableMethod();</code>	<pre>for (auto&& val : gen()) { val.mutableMethod(); }</pre>
<code>consume(func());</code>	<code>ranges::for_each(gen(), consume);</code>
<pre>auto var = func(); use(var); consume(std::move(var));</pre>	<pre>for (auto&& var : gen()) { use(var); consume(std::move(var)); }</pre> // Note: for generator<T> specifically, // is is safe to move from an auto&& // induction variable, even if that // isn't valid for all ranges
<pre>auto&& var = func(); use(var); consume(std::forward<decltype(var)> (var));</pre>	<pre>for (auto&& var : gen()) { use(var); consume(std::forward<decltype(var)> (var)); }</pre>

Using the model from section 4.1 of [P1362R0], a `generator<T> function()` is conceptually like a plain-old function that synchronously returns a `T`, except rather than returning a value once, it returns a sequence of values computed on demand. When a function returns a `T` I don't have to pay for a copy to assign it to a variable, and that should work even if it is move only. Because it is providing me with my own `T`, I should be free to `move()` it along to where I want it to go.

I should also be free to mutate the return value. I know some would prefer a `const`-by-default language, but that isn't where C++ is. `func().mutableMethod()` is specifically made legal in C++, even when `func()` returns a prvalue. I think it is inconsistent with that for [P2502R0] to propose that `generator<T>` disallow mutating operations on the results.

If the API for consuming a range looked more like `optional<T> next()`, then we could just return a `T` by value from that function and be done. For better or worse, the API for `input_range` differs from that API in two key ways: 1) the functions to get the current value and advance the current position are separated, and 2) if the current position isn't changed, it is required that *it* can be called multiple times and return equal results. This largely rules out returning `T` here, unless it is possible and cheap to copy.

For callers, `f()` generally behaves the same regardless of whether it returns `T` or `T&&` because in both cases the expression is an rvalue. The main exceptions are the result of `decltype(f())` introspection,

and that unqualified `T` returns are more useful for immobile types.

§ 3.3 Best match for the `generator<T>` use cases

`generator<T>`, as the name implies is a great tool when you want to generate some `T` objects. When making new `T` objects it is wasteful to have the caller copy them. `generator<T>` isn't going to be the best tool for making iterators over existing data structures, because it is limited (by necessity) to `input_range` while data structure iterators should at least provide `forward_range`. Additionally, even if you were OK with that limitation, you would probably want to yield `T&` rather than `const T&`, so you could use `generator<T&>` which this paper isn't proposing to change.

While we haven't used a coroutine-based generator in our production code base¹, we use a similar pattern in many places. Rather than STL-style iterators, code that generates a sequence of values implements an (often `virtual`, but not always) `optional<T> getNext()` method. Our experience is that nearly everything returned by these functions are either rvalues, or are locals that would be safe to move from (and with the latest rules will be implicitly moved). The initial plan to adopt that API also allowed `T*` return if you wanted to output objects owned by the callee but as far as I know, nobody has ever tried to use that option.

§ 3.4 Simplicity

The primary point of this proposal can be stated simply as `generator<FirstArg>::iterator::operator*()` should always return `FirstArg&&`. There are no exceptions or special cases. This uses the existing reference collapsing rules in the language to ensure it does the right thing in all cases, including when `FirstArg` is a reference type. This simplicity also carries into the implementation. The end of this paper includes an example implementation which does not need any use of `conditional_t`.

For comparison, [P2502R0] uses the following definition of its reference type:

```
conditional_t<is_void_v<U>,
    conditional_t<is_reference_v<T>, T, const T&>,
    T>
```

While not the ultimate decider, I think we should tend to favor designs without exceptions or special cases.

§ 4 Responses to objections

§ 4.1 Returning an rvalue reference from `*it` violates `input_or_output_iterator` semantic requirements

Specifically, the argument was that it violates the requirement that `*it` be equality preserving. This argument not true, because `*it` itself is equality preserving, it is only some *usages* of that expression that are not. And there is no requirement that, eg, `auto copy = *it;` be equality preserving.

While this was at one time a matter for debate, I think it is now settled that rvalue ranges *are* in fact valid ranges and at least valid `input_ranges`. [P2520R0] argues that they are also valid `forward-or-better` ranges, which is still under debate, however, that concern doesn't apply here because `generator` can only ever be an `input_range`. Assuming that [P2446R1] (`views::move`) lands, I think

rvalue-returning ranges will no longer be a rare edge case, and will be something that anyone writing range algorithms will need to handle anyway.

§ 4.2 Even if valid, rvalue-returning ranges are unsafe to consume

Any generic algorithm that works with ranges or iterators (for now, let's restrict this to `input_range` and `input_iterator`) should be prepared to handle rvalue-returning ranges, or add additional constraints to disallow them. This is already the case in the standard library. If any algorithms that accept such ranges or iterators do not meet their contract, then they have a bug.

But what about “normal users” who can't be expected to follow all of the guidelines for writing generic range algorithms, and just want to get their job done? Luckily for them, most things they do should still be safe! I will consider the 3 ways the people will consume generators separately, in the order of my best guesstimate of frequency that they will be used.

§ 4.2.1 Range-for loops

I expect this to be >90% of how `generator` is consumed.

Range-for loops are *always* safe to use with rvalue-returning ranges, because the name that is visible to users is an lvalue (even if it is an rvalue reference). That means there will be no implicit moves within the body of the loop. The only additional moves are:

1. In `for (auto val : gen) {...}` the generated values will be moved into `val` rather than copied. This is safe, and ideal because it avoids a performance trap. Admittedly, one that should be avoided in general by always using `auto&&` in range-for loops.
2. In `for (auto&& ref : gen) { consume(std::forward<decltype(ref)>(ref)); }` the object referred to by `ref` will be moved into `consume()`. But that isn't really an “implicit” move, and I think better reflects the intent of this code.
3. `for (auto&& ref : gen) { consume(std::move(ref)); }` will move into `consume()` with either a `T&&` or a `T&` reference type, but not with `const T&`. The other two cases only move with `T&&`.

[P2502R0] incorrectly claims they are unsafe even when used with range-for loops:

“ Using `T&&` as the default: This avoids a copy when doing `auto object = *it` (where `it` is a `std::generator::iterator`), but it is very easy to misuse, consider:

```
auto f = []() -> std::generator<std::string> { co_yield "footgun"; }();
for (auto&& x : f) {
    auto y = x; // nothing suggest a move
    y.transform();
    if (x != y) {
        // always triggers, likely to be surprising
    }
}
```

We rejected this option.

However, this analysis is incorrect. There is no move on the line `auto y = x;`. As explained above, the expression `x` is an lvalue, even if its declared type is `string&&`. Separately, I don't see why it would be surprising that `x != y` after `y` was modified, so I'm not sure what this example is trying to show. I think this is the only part of the paper that argues against `T&&`.

§ 4.2.2 Passing to std algorithms

I expect this to be ~9% of how `generator` is consumed.

As discussed above, the generic algorithms already need to be prepared to work with rvalue-returning input ranges.

One possible counter example is something like `views::filter([](string byValue) {...})`. This isn't a case of `filter` behaving incorrectly, it is a case of the user passing a predicate function that does not meet the [semantic requirements](#) of `predicate<string&&>`. If we wish to prevent this issue, I think it would be better to require algorithms to pass lvalues (ideally `const`-qualified, but that may be a breaking change) to predicates. That would solve the problem in general, while making `generator<T> yield const T&` would only solve it for generators.

Additionally, it seems possible that `rvalue_returning_input_range | views::filter(...)` | `to<vector>` may run afoul of [\[range.filter.iterator\]/1](#), since moving from an object may modify it in a way that it would no longer match the predicate. However, that paragraph both seems to both be insufficient to achieve its goals (it doesn't protect against modifications to the base range before the cached first match) and is underspecified (it doesn't say when the predicate would logically be evaluated, and if you assume it is at every modification, that leads to some nonsensical results). So I think there are some underlying specification issues there. I hope we can address that, at least for `input_ranges`, separately from consideration of `std::generator`.

§ 4.2.3 Using the iterators directly

This is where users *could* have issues. In particular, if they are passing `*it` multiple times to functions that accept a `T` by value or have a `T&&` overload.

```
use(*it); // moves
use(*it); // already moved from!
```

However, many common patterns avoid that already.

```
// Pretend the code also checks end() where needed. Also, assume that each chunk
// of code is in its own loop with its own iterator.

use(*it++); // moves, but that is good!

auto val = *it; // move here
use(val); // no move
use(val); // no move
use(move(val)); // explicit move from val

auto&& ref = *it; // no move or copy
use(ref); // no move
use(ref); // no move
use(FWD(ref)); // move from object referred to by ref and *it
```

```
it->func(); // never calls &&-qualified overload (unlike `(*it).func()`)
use(it->mem); // never calls &&-qualified overload (unlike `use((*it).mem)`)
```

All of those patterns are safe, and widely used already. And I think we should start treating multiple calls to `*it` as an anti-pattern that people should be taught to avoid anyway. It assumes that the expression `*it` is cheap or free, but that isn't necessarily the case. For example, `views::transform(expensive)` will invoke `expensive(*base_iterator)` on every call to its `*it`.

Additionally, I hope that we can relegate raw iterator usage to an expert or upper-intermediate level feature. At least in the code that I see people writing, we got 90% of the way there in C++11 with range-for loops. C++20 brings us closer to 99% since you no longer need to touch iterators in order to use algorithms like `sort`. I very rarely see “normal developers” write code that actually iterates iterators anymore rather than passing them along. Even among experts, it can be a once-in-a-while thing (of course, this does not apply to people working on standard libraries...)

§ 4.3 Implicitly moves in body of generator

Some people were concerned about implicit moves in the body of the generator function. However, at least as I am proposing it, that will not happen. I agree that it would be broken if it did implicitly move.

```
generator<string> exes() {
    string out = "x";
    while (true) {
        co_yield out; // will copy not move!
        out += 'x';
    }
}
```

That function would not perform its intended task if `co_yield out` moved from `out`. Of course, this function might be better returning a `generator<const string&>` so that the caller copies if it needs to, but it is *correct* either way.

§ 4.4 Implicitly copies in body of generator

I know some people have expressed concern about how `co_yield lvalue` will implicit copy. However, this is consistent with what happens in many other parts of the language so people should be expecting it. I would be more surprised if something that was yielding an unqualified `T` *didn't* copy when passed an lvalue.

```
T lvalue;
void consume(T);

void f() {
    consume(lvalue); // copies
    consume(move(lvalue)); // avoids copy

    T local;
    consume(local); // copies
}

T f() {
    return lvalue; // copies
    return move(lvalue); // avoids copy
}
```



```

    T local;
    return local; // no copy because we know `local` is going out of scope
}

// As I propose:
generator<T> f() {
    co_yield lvalue; // copies
    co_yield move(lvalue); // avoids copy

    T local;
    co_yield local; // copies, which is good!
    co_yield local; // unfortunately also copies.
}

```

The last line seems to be the only unfortunate case. I’m not proposing it at this time, but if we find that this is a common problem, we could consider a core-language change in the future to allow `co_yield` of an [implicitly movable entity](#) that is about to go out of scope to implicitly move with similar rationale as for `return`, `co_return`, and `throw`.

Also, in the inverse to the next point, the return type is by necessity both close to and (usually) under the control of the author of the generator function. `generator<T>` should be used when you can provide the consumer with their own `T` objects. If you don’t want to do that, then it is easy for the function author to choose to return a `generator<T&&>` or `generator<const T&>` depending on whether they want the consumer to be allowed to modify the yielded objects.

§ 4.5 We have `generator<T&&>` for people that want moves and it is more explicit

There are really two similar arguments here. First, that if someone wants moves, they can just use `generator<T&&>`. The problem I have with this is that usually the caller is the one who would need the moves, but they are not necessarily in control of the signatures of the functions they call. They may be part of a third party library. Or they may just not want to do the audit to ensure that all other callers will be ok with it.

The second argument is that using `generator<T&&>` as the return type makes it no longer a “hidden” move. The problem here is that the return type is usually stated very far away from the call site, often in a separate file. This is doubly true for iterators which A) are an extra layer removed from the underlying range, and B) are often processed in separate generic functions. When you see code like `use(*it); use(*it);` it looks identical regardless of whether it came from a `generator<T>` or a `generator<T&&>`, so I don’t think the latter is actually any more explicit *where it matters*.

§ 5 Example implementation

This is an example implementation of what I am proposing. It does not support allocators, but adding support for them should be simple and orthogonal. It also does not support recursive yielding via `elements_of`, since I do not think that should be included in the main `std::generator` type (but that discussion isn’t the point of this paper). It is also available on [godbolt](#), along with some examples, if you want to try it out. The same examples with the reference impl from [\[P2502R0\]](#) are available [here](#) if you would like to compare (the move-only example is commented out because it doesn’t compile).

```

template <typename T, typename ValueType = std::remove_cvref<T>>
struct generator {
    using value_type = ValueType;
    using Pointer = std::add_pointer_t<T>;

    struct promise_type {
        std::suspend_always initial_suspend() {
            return {};
        }
        std::suspend_always final_suspend() noexcept {
            return {};
        }

        // This overload copies v to a temporary, then yields a pointer to that.
        // This allows passing an lvalue to co_yield for a
        generator<NotReference>.
        auto yield_value(const T& v) requires(!std::is_reference_v<T> &&
                                              std::copy_constructible<T>) {
            struct Owner : std::suspend_always {
                Owner(const T& val, Pointer& out) : v(val) {
                    out = &v;
                }
                Owner(Owner&&) = delete;
                T v;
            };
            return Owner(v, ptr);
        }

        auto yield_value(T&& v) { // lval ref if T is lval ref
            ptr = &v;
            return std::suspend_always();
        }

        auto get_return_object() {
            return generator(this);
        }

        void return_void() {}
        void unhandled_exception() {
            throw;
        }

        auto coro() {
            return std::coroutine_handle<promise_type>::from_promise(*this);
        }

        Pointer ptr = {};
    };

    generator(generator&& source) : p(std::exchange(source.p, nullptr)) {}
    explicit generator(promise_type* p) : p(p) {}
    ~generator() {
        if (p)
            p->coro().destroy();
    }

    using sentinel = std::default_sentinel_t;
    struct iterator {
        using iterator_concept = std::input_iterator_tag;
        using difference_type = ptrdiff_t;
        using value_type = std::remove_cvref_t<T>;

```

```

    bool operator==(sentinel) const {
        return p->coro().done();
    }
    iterator& operator++() {
        p->ptr = {};
        p->coro().resume();
        return *this;
    }
    void operator++(int) {
        operator++();
    }
    Pointer operator->() const {
        return p->ptr;
    }
    T&& operator*() const { // lval ref if T is lval ref
        return static_cast<T&&>(*p->ptr);
    }

    promise_type* p;
};
auto begin() {
    p->coro().resume();
    return iterator{p};
}
auto end() {
    return sentinel();
}

promise_type* p;
};

```

§ 6 References

- [P1362R0] Gor Nishanov. 2018-11-15. Incremental Approach: Coroutine TS + Core Coroutines.
<https://wg21.link/p1362r0>
- [P2168R3] Corentin Jabot, Lewis Baker. 2021-04-19. generator: A Synchronous Coroutine Generator Compatible With Ranges.
<https://wg21.link/p2168r3>
- [P2446R1] Barry Revzin. 2021-11-17. views::all_move.
<https://wg21.link/p2446r1>
- [P2502R0] Casey Carter. 2021-12-13. std::generator: Synchronous Coroutine Generator for Ranges.
<https://wg21.link/p2502r0>
- [P2520R0] Barry Revzin. 2022-01-16. move_iterator should be a random access iterator.
<https://wg21.link/p2520r0>

-
1. However, I have used the `generator<T>` implementation included in this paper in a long running POC↩